LEARN BEING FORWARD

Programming — C# Programming

Collections & LINQ

COMPLETE LEARNING GUIDE

What You'll Learn

Real programs spend an enormous amount of their logic querying, filtering, and transforming collections of data, and C# provides two complementary tools for this: the generic collections in System.Collections.Generic (List<T>, Dictionary<TKey,TValue>, and friends) for STORING data, and LINQ (Language Integrated Query) for QUERYING and transforming it with a declarative, SQL-like syntax that works uniformly across in-memory collections, databases, and XML. LINQ's deferred execution model — where a query describes WHAT to compute without actually running it until the results are iterated — is both one of its most powerful features and one of its most common sources of subtle bugs for developers who assume a query runs immediately like a regular method call. This sub-module covers the core generic collections, IEnumerable as the foundation LINQ builds on, both LINQ syntax styles, deferred execution, the most common query operators, and aggregation.

1. Arrays and List<T>

An array (int[] nums = new int[5];) has a fixed size set at creation and cannot grow or shrink afterward. List<T> is a generic, dynamically resizable collection built on top of an array internally, automatically growing its backing storage as elements are added via Add — it's the default general-purpose collection for an ordered, resizable sequence of items, directly analogous to C++'s std::vector<T>. Being generic (the <T> type parameter), List<int> and List<string> are both fully type-checked at compile time, with no boxing/unboxing overhead for value types and no risk of accidentally adding a mismatched type, unlike the older, non-generic ArrayList from early .NET versions.

CSHARP

```
List<string> names = new List<string> { "Asha", "Ravi" };
names.Add("Meera");
names.Remove("Ravi");

foreach (string name in names) {
    Console.WriteLine(name); // Asha, Meera
}
Console.WriteLine(names.Count); // 2
```

2. Dictionary<TKey, TValue> and Other Generic Collections

Dictionary<TKey, TValue> stores unique keys mapped to values with average O(1) lookup, insertion, and removal via hashing — directly analogous to C#'s own HashSet<T> (unique values, no mapping) and to C++'s unordered_map/unordered_set. Accessing a missing key with the indexer (dict["missing"]) throws a KeyNotFoundException, so checking first with ContainsKey or using TryGetValue (which returns a bool success flag and avoids the exception entirely) is the standard, safe way to handle keys that might not be present. Queue<T> (first-in-first-out) and Stack<T> (last-in-first-out) round out the standard generic collections for specific access-order needs.

```csharp
var ages = new Dictionary<string, int> { ["Asha"] = 30, ["Ravi"] = 25 };

if (ages.TryGetValue("Asha", out int age)) {
    Console.WriteLine($"Asha is {age}"); // Asha is 30
}

if (!ages.ContainsKey("Priya")) {
    Console.WriteLine("Priya not found"); // avoids a KeyNotFoundException
}
```

3. IEnumerable and Iterating Collections

IEnumerable<T> is the fundamental interface nearly every collection in .NET implements, exposing just enough (a GetEnumerator method) to support foreach iteration — it's the common foundation that lets foreach, and LINQ itself, work identically across arrays, lists, dictionaries, and any custom collection type, without needing to know the concrete collection type at all. This is directly analogous to the C++ STL's iterator-based uniformity across different container types, and is precisely why LINQ's extension methods (Where, Select, and the rest) are defined once on IEnumerable<T> and then automatically available on every collection type that implements it.

4. LINQ Query Syntax vs. Method Syntax

LINQ offers two equivalent syntaxes for the same underlying operations: query syntax reads like a declarative SQL-style sentence (from x in collection where condition select x), and method syntax chains extension methods with lambda expressions (collection.Where(x => condition)) — both compile down to the exact same method calls, and most real-world C# code uses method syntax for its conciseness and composability, reserving query syntax mainly for queries with multiple joins or groupings where the SQL-like structure genuinely reads more clearly.

```csharp
var numbers = new List<int> { 1, 2, 3, 4, 5, 6 };

// Query syntax
var evensQuery = from n in numbers where n % 2 == 0 select n;

// Method syntax — equivalent, and far more common in practice
var evensMethod = numbers.Where(n => n % 2 == 0);

Console.WriteLine(string.Join(", ", evensMethod)); // 2, 4, 6
```

5. Deferred Execution: The LINQ Gotcha

Most LINQ operators (Where, Select, OrderBy) don't actually run when the query is WRITTEN — they build up a description of the computation and only execute it when the results are actually enumerated (by a foreach, a call to ToList()/ToArray(), or similar) — this is deferred execution. The practical danger: if the underlying collection changes between when a deferred query is defined and when it's finally enumerated, the query sees the collection's state AT ENUMERATION TIME, not at definition time, which can produce surprising results (or an exception, if the collection was modified in a way that invalidates iteration) if you assumed the query had already 'captured' a snapshot. Calling .ToList() immediately forces eager evaluation, materializing the results into a concrete list right away and avoiding this surprise.

```csharp
var numbers = new List<int> { 1, 2, 3 };
var query = numbers.Where(n => n > 1); // NOT executed yet — just describes the filter

numbers.Add(4); // modifies the list BEFORE the query is enumerated

foreach (int n in query) {
    Console.Write(n + " "); // 2 3 4 — the query sees the list AS IT IS NOW, including the 4
}
Console.WriteLine();

var eager = numbers.Where(n => n > 1).ToList(); // forces immediate evaluation, a real snapshot
```

6. Common Query Operators: Where, Select, OrderBy, GroupBy

Where filters a sequence to elements matching a predicate; Select projects (transforms) each element into a new form, often a different type entirely; OrderBy/OrderByDescending sorts by a key selector; GroupBy partitions a sequence into groups sharing a common key, producing a sequence of groups each with a Key property and its own enumerable of matching elements. These compose freely by chaining, which is the core strength of LINQ's design — a single readable chain (collection.Where(...).Select(...).OrderBy(...)) replaces what would otherwise be several nested loops with manual accumulator variables.

```csharp
var people = new List<(string Name, int Age)> {
    ("Asha", 30), ("Ravi", 25), ("Meera", 30), ("Jay", 25)
};

var groupedByAge = people.GroupBy(p => p.Age);
foreach (var group in groupedByAge) {
    Console.WriteLine($"Age {group.Key}: {string.Join(", ", group.Select(p => p.Name))}");
}
// Age 30: Asha, Meera
// Age 25: Ravi, Jay
```

7. Aggregation: Sum, Count, Average, Aggregate

Aggregation operators collapse a sequence down to a single value: Sum, Average, Min, and Max work directly on numeric sequences (or via a selector for sequences of complex objects), Count returns the number of elements (optionally matching a predicate), and Aggregate is the general-purpose fold operation for custom accumulation logic that doesn't fit one of the built-in aggregates. First() and Single() both return one specific element but differ critically in their failure behavior: First() returns the first match or throws if the sequence is empty (FirstOrDefault() returns a default value instead of throwing), while Single() throws if there is anything other than EXACTLY one match — using the wrong one for a given situation is a common source of unexpected exceptions.

```csharp
var numbers = new List<int> { 1, 2, 3, 4, 5 };

Console.WriteLine(numbers.Sum());                          // 15
Console.WriteLine(numbers.Average());                      // 3
Console.WriteLine(numbers.Count(n => n % 2 == 0));         // 2 (count matching a predicate)

int product = numbers.Aggregate(1, (acc, n) => acc * n); // custom fold: 1*1*2*3*4*5
Console.WriteLine(product); // 120

int firstEven = numbers.FirstOrDefault(n => n > 10); // no match -> default(int) = 0, no exception
Console.WriteLine(firstEven); // 0
```

Common Pitfalls

- Accessing a Dictionary key with the indexer without checking ContainsKey/TryGetValue first, risking a KeyNotFoundException on a missing key.
- Assuming a LINQ query executes immediately when written, instead of understanding deferred execution runs it only when enumerated.
- Modifying a collection while a foreach (or an un-materialized LINQ query) is actively iterating over it, risking an InvalidOperationException.

- Using First() when FirstOrDefault() was intended (or vice versa), causing an unexpected exception on an empty/no-match sequence.
- Re-enumerating the same un-materialized LINQ query multiple times, silently re-running the whole query (and any underlying data fetch) each time instead of caching the result with ToList().
- Using Single() when multiple matches are actually possible, triggering an exception instead of the intended 'just grab one' behavior that First() provides.

Key Takeaways

- List<T> and Dictionary<TKey,TValue> are the default generic collections for ordered sequences and key-based lookup, respectively.
- IEnumerable<T> is the common interface foreach and LINQ both build on, letting the same query logic work across any compliant collection type.
- LINQ query syntax and method syntax compile to identical calls; method syntax is far more common in everyday code.
- Most LINQ operators use deferred execution — the query runs only when enumerated, not when it's written; call .ToList() to force immediate evaluation.
- Choose First()/FirstOrDefault() vs. Single()/SingleOrDefault() deliberately based on whether more than one match is a valid, expected possibility.

Learn More

Microsoft Learn — LINQ (Language Integrated Query)
https://learn.microsoft.com/en-us/dotnet/csharp/linq/

Microsoft Learn — Collections (C#)
https://learn.microsoft.com/en-us/dotnet/standard/collections/